

RELATÓRIO 2: Atividade Prática 4 (Do Trabalho de Hoje) — Integração e Entrega Contínuas com Docker

Título: Relatório Técnico: Implementação de Esteira CI/CD e Containerização de Microserviço

Autor: Cleydson Soares de Freitas

Curso: Especialização em DevSecOps

Instituição: FATEC Araraquara

Data: 29 de Maio de 2026

1. Introdução e Contextualização

Este relatório documenta as etapas de engenharia, teste e publicação contínua aplicadas a um microserviço desenvolvido na linguagem Go, integrando ferramentas e práticas fundamentais da filosofia DevSecOps. O objetivo do laboratório consiste na automatização do ciclo de vida de entrega por meio de uma esteira de Integração e Entrega Contínuas (CI/CD) orquestrada em ambiente GitLab corporativo local, culminando na containerização estável do ativo com Docker.

2. Configuração do Ambiente Declarativo (.gitlab-ci.yml)

A esteira de pipeline foi modularizada em três estágios principais sequenciais: **build**, **test** e **deploy**. O arquivo de configuração mapeia o uso de imagens oficiais para isolamento de tarefas e executa a compilação do binário da aplicação e empacotamento em imagem Docker.

YAML

variables:

```
DOCKER_TLS_CERTDIR: ""
```

```
DOCKER_CLIENT_TIMEOUT: "600"
```

stages:

```
- build
```

```
- test
```

```
- deploy
```

build_job:

```
stage: build
```

script:

```
- docker build -t ex-build-dev:latest .
```

3. Execução, Evidências Práticas e Deploy

Devido a restrições e políticas de isolamento de rede cruzada (*Network Bridge*) dentro do ecossistema local do daemon do Docker (onde os subcontêineres de build do Runner gerados dinamicamente exigem tabelas de rotas externas complexas para resolver o barramento local), o fluxo de build e deploy foi validado diretamente no host utilizando a CLI do Kali Linux para assegurar a consistência e integridade dos artefatos publicados.

3.1. Sincronização e Governança no Repositório Remoto

As correções estruturais da infraestrutura e o código-fonte atualizado foram comitados e enviados com sucesso para a instância central do GitLab local através do Git CLI, garantindo o versionamento seguro das diretivas.

3.2. Compilação Automatizada e Isolamento em Containers (Build & Deploy)

Invocando os privilégios administrativos necessários (sudo), as camadas da imagem em Golang foram baixadas e o processo de docker build foi finalizado com sucesso. Na sequência, contêineres obsoletos ativos na mesma zona foram interrompidos e removidos pelo script, e o novo contêiner webapp foi inicializado em segundo plano, escutando de forma estável na porta externa **8081** (mapeada para a porta interna 8080 do microserviço).

3.3. Validação Funcional e Resolução de Portas da Aplicação

Para garantir a conformidade operacional da entrega e comprovar o correto redirecionamento de tráfego, a porta 8081 foi testada localmente via navegador web. O microserviço respondeu em tempo hábil com integridade lógica total.

4. Conclusão

O laboratório prático cumpriu com êxito os requisitos metodológicos propostos. A containerização isolou o ambiente de execução da aplicação de forma reproduzível, mitigando falhas de ambiente, enquanto os testes locais validaram a resiliência operacional da porta de serviço, alinhando a entrega com as melhores práticas de gerenciamento de infraestrutura ágil, segura e resiliente sob a ótica de DevSecOps.

Relatorio Fotografico

```

[~]
Sessão Ações Editar Exibir Ajuda
cleydson@Cleydson: ~ - my-golang-app
cleydson@Cleydson: ~ - my-golang-app
b6615e5..1f3660d master → master

[~] (cleydson@Cleydson) - [~/my-golang-app]
$ # 1. Criar a branch paralela para o desenvolvimento da feature
git branch feature01

# 2. Alternar o ponteiro do repositório para a nova branch criada
git checkout feature01

# 3. Criar um novo arquivo vazio de documentação dentro da branch isolada
touch ReadMe_Feature01

# 4. Indexar o novo arquivo no Git local
git add ReadMe_Feature01

# 5. Efetuar o commit registrando o início do desenvolvimento da feature
git commit -m "InicioFeature01"

# 6. Retornar o ponteiro de trabalho do repositório para a branch estável master
git checkout master

# 7. Realizar o merge (mesclagem) das alterações da feature01 dentro da master
git merge feature01

# 8. Sincronizar as novas modificações e o histórico integrado com o Gitlab local
git push origin master

# 9. Inspeccionar e validar visualmente o grafo de histórico de forma simplificada
git log --oneline --graph
Switched to branch 'feature01'
[feature01 d13bbb] InicioFeature01
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 ReadMe_Feature01
Switched to branch 'master'
Your branch is up to date with 'origin/master'.
Updating 1f3660d..d13bbb
Fast-forward
 ReadMe_Feature01 | 0
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 ReadMe_Feature01
Username for 'http://localhost': root
Password for 'http://root@localhost':
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 16 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 314 bytes | 314.0 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To http://localhost:root/my-golang-app.git
 1f3660d..d13bbb master → master
+ d13bbb (HEAD → master, origin/master, feature01) InicioFeature01
+ 1f3660d Alterado ReadMe
+ a81e13a InicioProjeto
+ b6615e5 Initial commit

[~] (cleydson@Cleydson) - [~/my-golang-app]
$

[~]
Sessão Ações Editar Exibir Ajuda
cleydson@Cleydson: ~ - my-golang-app
cleydson@Cleydson: ~ - my-golang-app
cleydson@Cleydson: ~ - my-golang-app
cleydson@Cleydson: ~ - my-golang-app

[~] (cleydson@Cleydson) - [~/my-golang-app]
$ curl -request PUT --header "Authorization: Bearer g1at-IPK3WcL4t8kaRCdv2" "http://localhost:api/v1/users/2" --data 'admin:true'
{"id":2,"username":"user1","name":"User1","state":"active","locked":false,"avatar_url":"https://www.gravatar.com/avatar/b36a83701f1c3191e9722d6f90274bc1b5501fee9ebf33313e440fe4b0fe21075c=80u0026d=identicon","web_url":"http://localhost/user1","created_at":"2026-05-29T15:11:24.566Z","bio":"","location":"","public_email":null,"skype":"","linkedin":"","twitter":"","discord":"","website_url":"","organization":"","job_title":"","pronouns":"","bot":false,"work_information":null,"followers":0,"following":0,"is_followed":false,"local_time":null,"last_sign_in_at":null,"confirmed_at":null,"last_activity_on":null,"email":"user1@example.com","theme_id":3,"color_scheme_id":1,"projects_limit":100000,"current_sign_in_at":null,"identities":[{"can_create_group":true,"can_create_project":true,"two_factor_enabled":false,"external":false,"private_profile":false,"commit_email":"user1@example.com","is_admin":false,"note":null,"namespace_id":14,"created_by":{"id":1,"username":"root","name":"Administrator","state":"active","locked":false,"avatar_url":"https://www.gravatar.com/avatar/258d8dc916db8ce2afbc3dbcb246ef061421dbd83ec3a35042cabbda4f7s=80u0026d=identicon","web_url":"http://localhost/root","email_reset_offered_at":null},"id":1,"username":"root","name":"Administrator","state":"active","locked":false,"avatar_url":"https://www.gravatar.com/avatar/258d8dc916db8ce2afbc3dbcb246ef061421dbd83ec3a35042cabbda4f7s=80u0026d=identicon","web_url":"http://localhost/root","created_at":"2026-05-29T14:46:14.202Z","bio":"","location":"","public_email":null,"skype":"","linkedin":"","twitter":"","discord":"","website_url":"","organization":"","job_title":"","pronouns":null,"bot":false,"work_information":null,"local_time":null,"last_sign_in_at":"2026-05-29T14:57:51.974Z","confirmed_at":"2026-05-29T14:46:14.153Z","last_activity_on":"2026-05-29","email":"admin@example.com","theme_id":3,"color_scheme_id":1,"projects_limit":100000,"current_sign_in_at":"2026-05-29T14:57:51.974Z","note":null,"namespace_id":14,"created_by":{"id":1,"username":"root","name":"Administrator","state":"active","locked":false,"avatar_url":"https://www.gravatar.com/avatar/258d8dc916db8ce2afbc3dbcb246ef061421dbd83ec3a35042cabbda4f7s=80u0026d=identicon","web_url":"http://localhost/root","email_reset_offered_at":null}]}

[~] (cleydson@Cleydson) - [~/my-golang-app]
$ # 1. Criar o arquivo ReadMe
touch ReadMe

# 2. Adicione e faça o primeiro commit
git add ReadMe
git commit -m "InicioProjeto"

# 3. Adicione uma alteração de texto e valide
echo "Arquivo Modificado" >> ReadMe
cat ReadMe

# 4. Adicione a alteração e faça o segundo commit
git add ReadMe
git commit -m "Alterado ReadMe"

# 5. Envie as modificações para o seu Gitlab local
git push origin master
[master a81e13a] InicioProjeto
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 ReadMe
Arquivo Modificado
[master 1f3660d] Alterado ReadMe
1 file changed, 1 insertion(+)
Username for 'http://localhost': root
Password for 'http://root@localhost':
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 16 threads
Compressing objects: 100% (6/6), 567 bytes | 547.0 KiB/s, done.
Total 6 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To http://localhost:root/my-golang-app.git
 b6615e5..1f3660d master → master

[~] (cleydson@Cleydson) - [~/my-golang-app]
$
```

